

SVEUČILIŠTE U ZAGREBU
FAKULTET ORGANIZACIJE I INFORMATIKE
V A R A Ž D I N

Gabriela Perković,
Mia Šižgorić,
Martina Vincijanović

SOKOBAN

SEMINAR IZ KOLEGIJA UVOD U MODELIRANJE ZNANJA

Varaždin, 2026.

SVEUČILIŠTE U ZAGREBU
FAKULTET ORGANIZACIJE I INFORMATIKE
V A R A Ž D I N

Gabriela Perković,

Mia Šižgorić,

Martina Vincijanović

Studij: Informacijski i poslovni sustavi, UIUP

SOKOBAN

SEMINAR IZ KOLEGIJA UVOD U MODELIRANJE ZNANJA

Mentorica:

Prof. dr. sc. Sandra Lovrenčić

Varaždin, siječanj 2026.

Gabriela Perković,

Mia Šižgorić,

Martina Vincijanović

Izjava o izvornosti

Izjavljujem da je moj završni/diplomski rad izvorni rezultat mojeg rada te da se u izradi istoga nisam koristio drugim izvorima osim onima koji su u njemu navedeni. Za izradu rada su korištene etički prikladne i prihvatljive metode i tehnike rada.

Autor/Autorica potvrdio/potvrdila prihvaćanjem odredbi u sustavu FOI-radovi

Sadržaj

Sadržaj.....	iv
1. Uvod.....	1
2. Objašnjenje igre Sokoban	2
2.1. Osnovna pravila i elementi ploče	2
2.2. Modeliranje stanja i operatora	3
2.3. Deadlock stanja	3
2.4. Razina težine i složenost problema	4
3. Rješenje u Prologu	6
3.1. Rješenje iz repozitorija	6
3.2. Objašnjenje poteza i rješenja	7
4. Rješenje u Pythonu.....	8
4.1. Objašnjenje ploče i simbola.....	8
4.2. Provjera poteza i guranje kutija	9
4.3. Provjera završetka razine	9
5. Usporedba rješenja u Prologu i Pythonu.....	11
6. Procjena prednosti i nedostataka rješenja	12
6.1. Prednosti i nedostaci Prolog rješenje	12
6.2. Prednosti i nedostaci Python rješenja	12
6.3. Zaključak procjene.....	12
7. Postavljanje upita chatbotu.....	13
8. Analiza odgovora chatbota	14
9. Zaključak.....	15
Popis literature.....	16
Prilog A – Izvorni kod Prolog rješenja	17
Prilog B – Izvorni kod Python rješenja	25

1. Uvod

Sokoban je početkom osamdesetih napravio Hiroyuki Imabayashi. To je logička igra koja na prvi pogled djeluje jednostavno: igrač pomiče kutije po skladištu i pokušava ih dovesti na ciljna polja. Međutim, već nakon nekoliko razina postaje jasno da se iza tih osnovnih pravila skriva složen problem planiranja i donošenja odluka. Upravo zbog toga Sokoban se često koristi kao primjer u umjetnoj inteligenciji, pretraživanju prostora stanja i formalnom modeliranju problema.

U okviru kolegija Uvod u modeliranje znanja Sokoban je posebno dobar odabir jer omogućuje da se razdvoje dvije ključne stvari. Prva je opis domene, odnosno jasna definicija pravila i ograničenja. Druga je postupak rješavanja, odnosno način na koji se iz definiranog modela dolazi do cilja pomoću rezoniranja i pretraživanja. Drugim riječima, ne zanima nas samo „igranje igre“, nego i precizan odgovor na pitanje kako taj problem formalno opisati tako da ga program može razumjeti i rješavati.

Da bi se Sokoban mogao rješavati sustavno, potrebno je jasno odrediti što uopće čini jedno stanje igre. U praksi to znači opisati položaj igrača i položaje kutija, zatim definirati koji su potezi dopušteni i što se točno događa nakon svakog poteza. Jednako je važno prepoznati cilj, odnosno stanje u kojem su sve kutije postavljene na predviđena ciljna polja. Tek kada je taj dio dobro postavljen, možemo smisleno usporediti različite pristupe rješavanju.

U ovom radu prikazana su dva pristupa koji se prirodno nadopunjuju: deklarativni pristup u Prologu i imperativni pristup u Pythonu. Prolog kao logički deklarativni jezik omogućuje da se pravila igre opišu kroz činjenice i pravila, a rješenje se dobije kroz zaključivanje i pretraživanje koje proizlazi iz samog modela [1], [2]. Python s druge strane traži da se provjera poteza i promjene stanja implementiraju eksplicitno, ali zato olakšava izradu demonstracijske aplikacije i povezivanje s grafičkim sučeljem, što je korisno za razumijevanje ponašanja igre i testiranje [3].

U radu se najprije se objašnjava logika igre i tipične situacije koje otežavaju rješavanje, zatim se prikazuju rješenja u Prologu i Pythonu te uspoređuju njihovi pristupi. Nakon toga obrađuje se dio vezan uz AI chatbot, uključujući postavljeni upit, sažetak odgovora i kritičku analizu njegove točnosti i korisnosti. Na kraju rada nalazi se popis literature.

Tijekom izrade ovog seminara rad je organiziran tako da svaka članica tima ima jednaku količinu posla. Analizu logike problema i opis problema izradila je Gabriela. Implementaciju rješenja u Prologu obradila je Martina, a Python implementaciju je obradila Mia. Usporedbu između Pythona i Prologa je napravila Gabriela, dok su upite i analizu AI chatbota odradile Martina i Mia.

2. Objašnjenje igre Sokoban

2.1. Osnovna pravila i elementi ploče

Sokoban se igra na ploči koja izgleda kao jednostavna mreža polja, ali se vrlo brzo pokaže da svaki element na toj ploči ima svoju ulogu. Najlakše je razinu zamisliti kao mali skladišni prostor omeđen zidovima. Unutar tog prostora nalaze se prolazna polja po kojima se igrač može kretati, zatim ciljne pozicije na koje je potrebno dovesti kutije, te same kutije i lik kojim upravljamo. U literaturi se ciljna polja često opisuju kao skladišna mjesta, a u pojedinim opisima i implementacijama nazivaju se i „dock“ pozicije [4], [5]. Zidovi i ciljevi su pritom stalni, dok se igrač i kutije mijenjaju kroz poteze.

Kretanje igrača je vrlo jednostavno definirano. Igrač se može pomaknuti za jedno polje u smjeru gore, dolje, lijevo ili desno, ali samo ako ispred njega nije zid. U tom dijelu Sokoban podsjeća na mnoge druge igre na mreži, no razlika nastaje u trenutku kada se ispred igrača pojavi kutija. Kutije su glavni „problem“ igre jer se ne ponašaju kao obične prepreke. Igrač ih može pomaknuti, ali samo guranjem, nikada povlačenjem, i to je ograničenje koje mijenja cijelu logiku rješavanja [4]. Guranje je dopušteno samo ako je kutija odmah ispred igrača i ako je polje iza kutije slobodno ili predstavlja cilj. Ako iza kutije stoji zid ili druga kutija, potez jednostavno nije moguć. Iz istog razloga nije moguće pogurati dvije kutije odjednom, pa dvije kutije u liniji postaju nepomičan „zid“ koji blokira prolaz [4].

Zidovi, iako su naizgled samo granice prostora, u Sokobanu često stvaraju zamke. Budući da se kutije ne mogu vući, pogrešno guranje zna biti nepovratno. Primjerice, ako se kutija progura na mjesto s kojeg je više nije moguće gurati u korisnom smjeru, razina može postati nerješiva. Zato kutije treba gledati kao pokretne prepreke koje se u svakom trenutku ili mogu legalno gurnuti, ili se ponašaju kao čvrsta blokada. Igrač kroz cijelu razinu zapravo stalno balansira između toga da si otvara prolaz i da kutije postavlja na mjesta s kojih ih kasnije može dovesti do cilja [4], [6].

Upravo zbog ovih pravila Sokoban ima specifičnu dinamiku. U svakom je trenutku jasno može li igrač napraviti pomak ili gurnuti kutiju, dok se cilj odnosi na konačni raspored svih kutija na ciljna polja. Zbog toga se rješenje rijetko pronalazi odmah, već zahtijeva planiranje unaprijed i izbjegavanje poteza koji se u početku čine korisnima, ali kasnije mogu zatvoriti prostor ili blokirati kutije.

2.2. Modeliranje stanja i operatora

Kako bi se Sokoban mogao rješavati sustavno, prvo je potrebno jasno definirati što uopće znači „stanje“ igre. U najjednostavnijem smislu, stanje obuhvaća informaciju gdje se nalazi igrač i gdje se nalaze sve kutije. To je ujedno i minimum podataka bez kojih se ne može odrediti koji su potezi u nastavku dopušteni. Zidovi i ciljna polja pritom ne mijenjaju svoje pozicije, pa ih možemo promatrati kao dio okoline koji je stalan, dok su igrač i kutije elementi koji se tijekom igre pomiču.

U formalnijem zapisu, stanje se često prikazuje kao par (P, B) . Oznaka P predstavlja poziciju igrača, dok B predstavlja skup pozicija svih kutija na ploči. Takav zapis je praktičan jer omogućuje da se svaka konfiguracija igre opiše jednoznačno, a zatim uspoređuje s drugim konfiguracijama, primjerice radi provjere je li neko stanje već posjećeno ili je li postignut cilj [5]. Ciljno stanje tada možemo opisati jednostavno: ono je postignuto kada se sve kutije iz skupa B nalaze na ciljanim poljima, odnosno kada nijedna kutija ne ostane izvan skladišne pozicije [1].

Nakon što je stanje definirano, sljedeći korak je opisati operatore, odnosno poteze koji mijenjaju stanje. U Sokobanu operator predstavlja jednu akciju igrača u jednom od četiri smjera. U praksi postoje dvije situacije koje nas zanimaju. U prvoj situaciji igrač se pomiče na susjedno polje bez guranja kutije, pa se u novom stanju mijenja samo P . U drugoj situaciji igrač gura jednu kutiju, pa se u novom stanju mijenjaju i P i skup B , jer se uz igrača pomiče i jedna kutija.

Svaki operator ima jasno određene preduvjete. Primjerice, običan pomak je dopušten ako je susjedno polje prolazno, dok je guranje dopušteno samo ako je odmah ispred igrača kutija, a polje iza te kutije je slobodno ili je ciljno polje. Ako ti preduvjeti nisu zadovoljeni stanje se ne smije promijeniti. Učinci operatora su također jednoznačni: kod običnog pomaka mijenja se pozicija igrača, a kod guranja se uz to mijenja i pozicija gurnute kutije.

Ovakav način modeliranja posebno dobro odgovara Prologu, jer se prijelazi između stanja mogu izraziti kao logička pravila. Umjesto da se ručno opisuje procedura korak po korak, u Prologu se može zapisati relacija koja kaže da iz stanja S slijedi stanje $S2$ ako je primijenjen legalan operator, čime se problem prirodno veže uz zaključivanje i pretraživanje [2].

2.3. Deadlock stanja

Sokoban je posebna igra zato što velik dio poteza ima posljedice koje se ne mogu jednostavno „poništi“. Igrač se može vratiti na staru poziciju, ali kutija koju je jednom pogurnuo na loše mjesto često više ne može biti vraćena u korisno stanje. Upravo zbog toga se u Sokobanu vrlo često pojavljuje pojam deadlocka. Deadlock je stanje u kojem razina postaje

nerješiva, bez obzira na to koliko se igrač kasnije trudi i koliko poteza još napravi. Drugim riječima, cilj možda i dalje izgleda „blizu“, ali je logički nedostižan.

Najjednostavniji i najpoznatiji primjer je takozvani „kutni deadlock“. Ako se kutija gurne u kut koji nije ciljno polje, ona se više ne može pomaknuti. Razlog je vrlo konkretan: kutija bi se mogla pomaknuti samo ako je igrač može gurnuti, a u kutu ne postoji smjer iz kojeg igrač može stati „iza“ kutije da bi je gurnuo van. Budući da vučenje nije dopušteno, kutija ostaje trajno zarobljena, a razina u pravilu postaje nerješiva [4].

Međutim, deadlock situacije nisu ograničene samo na kutove. Česte su i blokade uz zid, gdje kutija ostaje „zalijepljena“ uz zid na način da se kasnije više ne može dovesti na cilj, osobito ako ciljna polja nisu raspoređena duž te iste linije. Postoje i složeniji scenariji u kojima se više kutija međusobno blokira. Primjerice, dvije ili tri kutije mogu zauzeti položaje tako da svaka drugoj zatvara jedini smjer pomaka, pa nijedna više nije pomična u korisnom smjeru. Dodatno, kutije mogu zatvoriti prolaz kroz koji igrač mora proći da bi došao na stranu potrebnu za sljedeće guranje. U takvim situacijama problem nije samo u tome gdje su kutije, nego i u tome što igrač više ne može doći na poziciju s koje bi potez uopće bio moguć [7], [2].

Za ljudskog igrača deadlock se često prepoznaje kroz iskustvo. Nakon nekoliko pokušaja igrač počne „vidjeti“ koje pozicije kutija izgledaju sumnjivo i koje bi kasnije mogle uzrokovati blokadu. Za program je, međutim, deadlock još važniji jer automatsko pretraživanje bez provjere deadlocka može potrošiti ogromno vrijeme na grane pretraživanja koje su unaprijed osuđene na neuspjeh. Ako se deadlock prepozna dovoljno rano, takva grana se može odmah odbaciti, čime se prostor pretraživanja smanjuje i rješavanje postaje znatno učinkovitije [2].

2.4. Razina težine i složenost problema

Sokoban je jedna od onih igara koja se na početku čini gotovo trivijalnom. Igrač vidi nekoliko kutija i nekoliko ciljnih polja i intuitivno pomisli da je dovoljno samo „pogurati“ kutije na prava mjesta. Međutim, već nakon par pokušaja postane očito da se težina ne skriva u pravilima, nego u broju mogućnosti. Čak i s dvije ili tri kutije igra vrlo brzo nudi velik broj različitih poteza, a svaki potez otvara nove rasporede na ploči. Drugim riječima, broj mogućih stanja raste toliko brzo da se razina može pretvoriti u problem koji je teško riješiti metodom pokušaja i pogreške.

Ono što dodatno komplicira rješavanje jest činjenica da potezi nisu jednako „bezopasni“. Igrač se može vratiti korak unatrag, ali kutiju koju je jednom gurnuo na nezgodno mjesto često nije moguće dovesti natrag na korisnu poziciju. Ta nepovratnost je razlog zašto Sokoban traži planiranje. Potez koji u trenutku izgleda smisleno može kasnije zatvoriti prolaz, blokirati kutiju

uz zid ili stvoriti situaciju iz koje više nema rješenja. Zbog toga se rješavanje često svodi na procjenu: ne samo što trenutno mogu napraviti, nego i što time dugoročno gubim ili dobivam.

Kada se problem promatra računalno, situacija postaje još jasnija. Program koji pokušava riješiti Sokoban zapravo prolazi kroz prostor stanja, gdje je svako stanje određeno položajem igrača i rasporedom svih kutija. Ako se pretraživanje radi na površan način, program lako zapne u ponavljanju istih konfiguracija ili troši vrijeme na grane koje su unaprijed osuđene na neuspjeh. Upravo zato ozbiljniji pristupi uvode optimizacije. Jedna od osnovnih je pamćenje već viđenih stanja, kako se program ne bi vrtio u krug. Druga je uvođenje provjere deadlocka, čime se pokušava što ranije prepoznati situacije koje ne mogu voditi do rješenja. Uz to se koriste i naprednije strategije pretraživanja, koje su bolje od čistog pretraživanja u dubinu kada razine postanu složenije [2].

U ovom radu nećemo ulaziti u formalne dokaze složenosti, jer cilj nije matematičko dokazivanje nego razumijevanje modeliranja i pristupa rješavanju. Naglasak je na praktičnom prikazu kako se Sokoban može opisati kao problem sa stanjima i operatorima, te kako se taj isti problem može rješavati u dvjema različitim paradigmama. Upravo usporedba Prologa i Pythona najbolje pokazuje razliku između deklarativnog opisa domene i eksplicitne, imperativne implementacije postupka rješavanja [2].

3. Rješenje u Prologu

Prolog je logički, deklarativni programski jezik u kojem se program ne gradi kao niz naredbi koje se izvršavaju jedna za drugom, nego kao skup činjenica i pravila. Prilikom pisanja koda cilj je naglasiti što vrijedi u domeni. Rezultat se zatim dobiva postavljanjem upita, a Prolog pokušava pronaći odgovor koristeći pravila. Zbog takvog načina rada Prolog je posebno pogodan za probleme u kojima treba opisati domenu i prepustiti sustavu da kroz zaključivanje i pretraživanje pronađe rješenje [1], [2].

U slučaju Sokobana, ta prednost postaje vrlo praktična. Pravila igre mogu se zapisati kao skup logičkih uvjeta koji opisuju kada je potez dopušten i kako se stanje mijenja nakon tog poteza. Primjerice, prijelaz između stanja može se izraziti relacijom koja kaže da iz stanja S možemo dobiti novo stanje S2 ako igrač pomakne svoju poziciju u određenom smjeru i ako su zadovoljeni uvjeti kao što je na primjer „polje ispred nije zid“ ili „ispred je kutija, a iza nje je slobodno polje“. Na taj način je opisan kao pravilo koje povezuje početno i sljedeće stanje. Kada su pravila jednom definirana, Prolog može kroz pretraživanje automatski generirati slijed poteza koji vodi do cilja što se dobro uklapa u ideju modeliranja znanja u okviru ovog kolegija [1], [2].

3.1. Rješenje iz repozitorija

U ovom seminaru koristimo već kreirano Prolog rješenje iz GitHub repozitorija koje rješava Sokoban korištenjem pretraživanja u dubinu (DFS). Organizacija rješenja je pregledna i zapravo dobro odgovara načinu na koji se Sokoban prirodno modelira. Ideja je da se najprije definiraju pravila igre i opis ploče, a nakon toga se pokrene pretraživanje koje pokušava pronaći slijed poteza od početnog do ciljnog stanja [1].

Prema dokumentaciji repozitorija, središnja datoteka je sokoban.pl, koja sadrži logiku pretraživanja i učitava ostale dijelove programa. Pravila igre opisana su u game.pl. Dio vezan uz samu ploču i njezina pravila, kao što su dostupna polja i struktura prostora, nalazi se u datoteci board.pl. Razine su definirane u zasebnim datotekama, pri čemu svaka razina opisuje gdje su na početku igrač i kutije, koja su polja prolazna te koja su ciljna polja [1]. Takvo razdvajanje je korisno jer omogućuje da se ista pravila igre primjenjuju na različite razine bez mijenjanja logike pretraživanja.

Kada se program pokrene u SWI Prologu, učitava se željena razina i zatim se pozove predikat koji traži rješenje. Rezultat se prikazuje kao lista poteza, odnosno kao niz akcija koje vode do ciljnog stanja. U dokumentaciji je jasno naznačeno i da se po potrebi može uključiti dodatno ograničenje vezano uz to može li igrač uopće doći do pozicije s koje bi mogao gurati kutiju.

Takvo ograničenje ne mijenja pravila Sokobana, ali mijenja prostor pretraživanja i može smanjiti broj rješenja koja su u pravilu moguća, ali praktično besmislena [1].

3.2. Objašnjenje poteza i rješenja

U Prolog rješenju Sokobana rješenje se najčešće prikazuje kao lista poteza, odnosno kao niz akcija koje treba izvesti kako bi se iz početnog stanja došlo do ciljnog stanja. Svaki element liste predstavlja jedan konkretan potez i sadrži informaciju o tome koja se kutija gura i u kojem smjeru, ili se barem eksplicitno naznačuje smjer pomaka koji se u tom koraku događa.

Prema primjeru iz korištenog repozitorija, nakon učitavanja programa u SWI Prologu i poziva predikata za rješavanje, sustav vraća rezultat u obliku liste poteza. U dokumentaciji se vidi da se rješenje može pojaviti i u više varijanti, na primjer zbog drugačijeg redoslijeda guranja kutija ili zbog načina na koji DFS pretražuje prostor stanja [1]. To se događa jer postoji više mogućih putova do cilja, a algoritam pronalazi jedno ili više rješenja.

Sama igra funkcionira na način da se na početku definira početno stanje pri čemu se određuje početna pozicija kutija, ciljne pozicije i početna pozicija Sokobana odnosno igrača. Model stanja definiran je kombinacijom Sokoban pozicije i pozicija na kojima se nalaze kutije.

Ploča igre opisana je pravilima koja definiraju susjedstvo između polja, na primjer pravilo:

$$neib(P1, P2, right) :- right(P1, P2).$$

označava da se polje P2 nalazi desno od polja P1. Slična pravila postoje i za ostale smjerove kretanja. Kretanje se realizira prema jasno definiranim pravilima. Potez je dopušten ako Sokoban može pristupiti kutiji, ako postoji susjedno polje u zadanom smjeru i ako je provjera sljedećeg polja za kutiju uspješno izvršena, odnosno polje je slobodno i ne predstavlja prepreku.

Ciljno stanje se provjerava rekurzivno tako da se za svaku kutiju ispituje nalazi li se na odgovarajućoj ciljnoj poziciji. Ako su sve kutije na ispravnim mjestima, stanje se smatra zadovoljavajućim i rješenje je pronađeno.

4. Rješenje u Pythonu

Python je u ovom radu odabran prvenstveno zbog praktičnosti. Radi se o jeziku koji je čitljiv, široko korišten i dovoljno fleksibilan da se uz relativno malo koda može dobiti program koji je jednostavan za testiranje i demonstraciju. Kod Sokobana je to posebno korisno jer pravila igre postaju puno jasnija kada se vide „u pokretu“, a ne samo kao apstraktni opis. U korištenoj implementaciji igra je povezana s grafičkim prikazom putem biblioteke pygame, pa se razina može interaktivno igrati i promatrati, što olakšava razumijevanje mehanike i provjeru ispravnosti poteza [3].

Za razliku od Prologa, Python ne nudi ugrađene mehanizme logičkog zaključivanja i automatskog pretraživanja rješenja. To znači da se u Python pristupu sve ključne provjere i promjene moraju eksplicitno programirati. Potrebno je osmisliti kako će se stanje igre spremati, kako će se ažurirati nakon svakog poteza, te kako će se kontrolirati je li potez uopće dopušten. Uz to, potrebno je implementirati pravila guranja kutija i jasno definirati uvjete pod kojima se smatra da je razina riješena, primjerice kada više ne postoji nijedna kutija izvan ciljnih polja [3].

4.1. Objašnjenje ploče i simbola

U korištenoj Python implementaciji Sokobana razina je predstavljena kao matrica znakova, odnosno lista redaka u kojoj svaki znak opisuje jedno polje na ploči. Takav pristup je vrlo praktičan jer se stanje može jednostavno učitati iz tekstualne datoteke, lako se ispisuje radi provjere, a istu reprezentaciju je jednostavno povezati s grafikom, tako da se svaki znak mapira na odgovarajuću sličicu na ekranu [3].

Znakovi imaju jasno definirano značenje. Znak „#“ predstavlja zid i označava polje koje je uvijek neprolazno. Razmak „ “ predstavlja obično podno polje po kojem se igrač može kretati. Znak „@“ označava igrača kada stoji na podu, dok znak „+“ označava igrača kada se nalazi na ciljnom polju. Ciljna polja označena su znakom „.“. Kutija koja nije na cilju označena je znakom „\$“, a kutija koja je postavljena na cilj označena je znakom „*“ [3].

Ovakva implementacija ima nekoliko prednosti. Prvo, čitavo stanje razine vidljivo je „na prvi pogled“ i može se lako ručno čitati, što je korisno i za izradu vlastitih razina. Drugo, promjene stanja nakon poteza svode se na zamjenu znakova u matrici, što dodatno pojednostavljuje implementaciju. Treće, u grafičkom prikazu je dovoljno za svaki znak nacrtati odgovarajuću sliku, pa se logička reprezentacija i vizualizacija međusobno nadopunjuju [3].

4.2. Provjera poteza i guranje kutija

Kako bi igra funkcionirala program mora za svaki pritisak tipke odlučiti je li potez dopušten i, ako jest, kako točno treba promijeniti stanje ploče. U korištenoj Python implementaciji ta provjera radi se kroz metode koje najprije „pogledaju“ sadržaj polja ispred igrača, a zatim odlučuju radi li se o običnom pomaku ili o situaciji u kojoj se pokušava gurnuti kutija [3].

Ako se ispred igrača nalazi zid, potez se odmah odbacuje jer zid predstavlja trajno neprolazno polje. Slično tome, ako se ispred nalazi kutija, igrač se ne može samo pomaknuti na to polje kao da je prazno. U tom slučaju program provjerava još jedno polje u istom smjeru, odnosno polje iza kutije. Guranje je dopušteno samo ako je to polje slobodno ili ako je ciljna pozicija. Ako iza kutije stoji zid ili druga kutija, potez nije dopušten i stanje se ne smije promijeniti. Time se izravno provode osnovna pravila Sokobana; specifično pravilo da se kutije mogu gurati samo jednu po jednu i samo kada postoji prostor za pomak [3].

Kada je guranje dopušteno, promjena stanja nije samo „pomak igrača“, nego se mora ažurirati i položaj kutije. Program tada mijenja znakove u matrici tako da se igrač prebaci na polje gdje je bila kutija, a kutija se prebaci na sljedeće polje. Pritom je važno razlikovati obična polja i ciljna polja, jer se u simboličkoj reprezentaciji drugačije označava kutija na cilju („*“) i igrač na cilju („+“). Upravo zbog toga se u implementaciji pojavljuje više slučajeva koji na prvi pogled izgledaju kao ponavljanje, ali su potrebni kako bi se korektno održavala informacija o tome stoji li nešto na običnom polju ili na cilju [3].

Za potrebe ovog seminara važno je naglasiti da je prikazano Python rješenje u ovom obliku prvenstveno namijenjeno demonstraciji mehanike igre i interaktivnom igranju. Drugim riječima ono ne traži samo rješenje automatski, nego omogućuje korisniku da ručno rješava razinu uz provjere „legalnosti“ poteza. Jednaka reprezentacija stanja i ista pravila prijelaza mogu poslužiti kao temelj za automatsko rješavanje. Dovoljno bi bilo dodati algoritam pretraživanja prostora stanja, primjerice BFS za pronalazak najkraćeg rješenja ili A* za učinkovitije usmjeravanje pretraživanja pomoću heuristike, pri čemu bi se postojeće provjere poteza i ažuriranje stanja mogle ponovno iskoristiti.

4.3. Provjera završetka razine

Da bi program znao kada je razina završena, mora postojati jasan kriterij uspjeha. U Sokobanu je cilj postići situaciju u kojoj su sve kutije postavljene na ciljna polja. U simboličkoj reprezentaciji korištenoj u ovoj Python implementaciji to se može provjeriti na vrlo izravan način. Kutija koja još nije na cilju označena je znakom „\$“, dok je kutija koja je postavljena na cilj označena znakom „*“. To znači da je razina riješena upravo onda kada na ploči više ne postoji nijedna kutija označena s „\$“ [3].

U praksi se provjera radi tako da program prolazi kroz cijelu matricu znakova i traži pojavljivanja simbola „\$“. Ako se taj simbol pronađe, to znači da barem jedna kutija još uvijek nije na cilju i razina nije završena. Ako se pri pretraživanju matrice ne pronađe nijedan „\$“, program zaključuje da su sve kutije na ciljevima i razina se smatra riješenom. U tom trenutku implementacija može korisniku prikazati poruku o uspješno završenoj razini ili jednostavno onemogućiti daljnje pomicanje [3].

Ovakav kriterij je jednostavan, ali pouzdan, jer se oslanja na već postojeću reprezentaciju stanja. Drugim riječima, ne uvodi se dodatna struktura podataka za praćenje broja kutija na cilju, nego se koristi činjenica da se stanje već u svakom potezu održava konzistentnim kroz odgovarajuće simbole. Cijeli kod nalazi se u prilogu.

5. Usporedba rješenja u Prologu i Pythonu

Kad se isti problem pokuša riješiti u Prologu i u Pythonu, vrlo brzo se vidi da razlika nije samo u sintaksi, nego u načinu razmišljanja. Sokoban je tu dobar primjer jer se može opisati kao skup pravila, ali se može i implementirati kao „klasična“ igra u kojoj program u svakom trenutku mora znati što je na ploči i što je dopušteno napraviti.

Prolog pristup je bliži ideji modeliranja znanja. Najprije se opiše domena, odnosno pravila igre, a zatim se od sustava traži da na temelju tih pravila pronađe rješenje. U praksi to izgleda tako da se definiraju legalni potezi i uvjeti ciljnog stanja, a Prolog kroz pretraživanje i povratak unazad sam pronalazi put od početne do ciljne konfiguracije [1], [2]. Prednost je što je kod često kraći i jasnije pokazuje logiku problema. Nedostatak je što takav pristup, posebno kada se koristi jednostavan DFS, može brzo postati spor na složenijim razinama, jer se lako upadne u grane koje dugo ne vode nikamo [1].

Python rješenje ide iz suprotnog smjera. Umjesto da se oslanja na zaključivanje jezika, program mora eksplicitno provjeravati svaku situaciju i odlučivati što se smije dogoditi. To znači da se ručno implementira spremanje stanja ploče, provjera poteza i ažuriranje matrice nakon pomaka ili guranja. Taj dio je „teži“ za napisati, ali daje veću kontrolu i često je lakši za pratiti onima koji su navikli na imperativno programiranje. Osim toga, Python se jednostavno povezuje s grafikom, pa je prirodno da se ovdje rješenje koristi kao interaktivna demonstracija igre i njenih pravila, što je u edukativnom smislu jako korisno [3].

Razlika se dobro vidi i u cilju samih implementacija. Polog rješenje iz repozitorija je rješavač, njegov zadatak je pronaći listu poteza koja vodi do cilja, i to je fokus cijelog programa [1]. Python implementacija je prvenstveno igra, korisnik je taj koji donosi odluke, a program samo provjerava jesu li potezi legalni i prikazuje rezultat [3]. Naravno, Python verzija se može nadograditi tako da i sama traži rješenje, ali za to bi trebalo dodati algoritam pretraživanja, dok je u Prologu taj dio već prirodno uklopljen u način rada jezika.

U kontekstu kolegija, ova usporedba je korisna jer pokazuje da isti problem može izgledati „drugačije“ ovisno o alatu. Prolog nas gura prema pravilima i relacijama, a Python prema proceduri i kontroli toka. I jedno i drugo ima smisla, ali naglasak je drugačiji, pa se i pristup rješavanju mijenja [1], [2].

6. Procjena prednosti i nedostataka rješenja

6.1. Prednosti i nedostaci Prolog rješenje

Prednosti:

- prirodno modeliranje znanja kroz pravila i činjenice
- elegantno pretraživanje uz povratak unazad
- jasno razdvajanje domene i pretraživanja [1], [2]

Nedostaci:

- DFS može biti spor i može generirati velik broj nepotrebnih stanja
- bez dodatnih ograničenja moguće je ponavljanje rješenja
- Prolog može biti teži za početnike

6.2. Prednosti i nedostaci Python rješenja

Prednosti:

- jednostavna i čitljiva implementacija
- lako dodavanje grafike i interakcije
- praktično za demonstraciju načina funkcioniranja igre [3]

Nedostaci:

- sve se implementira ručno
- bez dodatnog algoritma pretraživanja nije automatski rješavač
- kompleksnost rješavanja se mora posebno adresirati ako se želi automatsko rješenje [3], [2]

6.3. Zaključak procjene

Ako je cilj naglasiti modeliranje znanja i zaključivanje, Prolog je prirodniji izbor. Ako je cilj izraditi praktičnu aplikaciju i vizualnu demonstraciju, Python je u tom slučaju bolji izbor. Tipično se koristi kombinacija formalnog modeliranja i pretraživanja uz heuristike te praktična implementacija sučelja.

7. Postavljanje upita chatbotu

Za dio seminara vezan uz AI chatbot postavljen je upit sustavu Gemini s ciljem dobivanja objašnjenja logike problema Sokoban i navođenja literature. Upit je glasio: „Objasni logiku problema Sokoban i navedi literaturu koju si koristio prilikom generiranja odgovora.“ (poveznica na razgovor dostupna je u literaturi) [8].

Cilj ovog upita bio je provjeriti može li AI alat točno objasniti pravila i logiku Sokobana i identificirati neke od izazova poput deadlocka. Također je bilo bitno za provjeriti koliko pouzdano navodi izvore koji potkrepljuju tvrdnje.

8. Analiza odgovora chatbota

AI chatbot je u odgovoru naveo sljedeće ključne točke:

- Sokoban je problemska domena na mreži u kojoj igrač pomiče kutije isključivo guranjem (bez mogućnosti povlačenja).
- Stanje igre može se modelirati položajem igrača i skupom položaja kutija, uz fiksne zidove i ciljna polja.
- Operator (akcija) definiran je pomakom u jednom od četiri smjera, pri čemu je guranje moguće samo ako je polje iza kutije slobodno ili ciljno.
- Istaknuta je važnost deadlock stanja (npr. kutni deadlock), jer mogu učiniti razinu nerješivom.
- Navedeno je da prostor stanja brzo raste i da rješavanje zahtijeva pretraživanje (npr. DFS/BFS) te eventualno heuristike za učinkovitije rješavanje.
- Kao dodatak, chatbot je naveo nekoliko izvora koji se često spominju u literaturi o Sokobanu.

Odgovor AI chatbota bio je u osnovi točan u opisu pravila igre, posebno u naglašavanju da se kutije guraju, a ne vuku, te u isticanju deadlock situacija. Također je spomenuo da je problem računalno složen te naveo nekoliko znanstvenih izvora koji se često spominju u literaturi o Sokobanu.

Iako je sve što je AI chatbot naveo bilo točno, postojala su određena važna ograničenja. AI chatbot nije precizno povezao tvrdnje s konkretnim izvorima literature. Iz tog razloga takav popis izvora bi se trebao zamijeniti provjerenim izvorima.

U ovom radu AI odgovor se koristi kao pomoćni materijal, ali svi ključni dijelovi objašnjenja potkrijepljeni su provjerljivim izvorima navedenim u literaturi. Može se zaključiti da AI chatbot može biti koristan kako bismo dobili početno razumijevanje teme, ali ne može zamijeniti stvarno čitanje izvora i pravilno citiranje. Stoga je nužno pružiti vlastiti kritični osvrt na točnost i način navođenja izvora u odgovoru pruženom od strane chatbota.

9. Zaključak

Sokoban je odličan primjer problemske domene u kojoj se jasno vidi razlika između jednostavnih pravila i složenosti rješavanja. Modeliranje znanja u ovoj domeni zahtijeva preciznu definiciju stanja, operatora i cilja, kao i razumijevanje situacija koje vode do nerješivosti, osobito deadlock stanja.

Usporedba Prologa i Pythona pokazuje kako programska paradigma usmjerava rješavanje. Prolog omogućuje prirodno izražavanje pravila i pretraživanje kroz zaključivanje, što je posebno relevantno za kolegij Uvod u modeliranje znanja [1], [2]. Python omogućuje praktičnu implementaciju i vizualnu demonstraciju, ali zahtijeva eksplicitno programiranje logike poteza i, za automatsko rješavanje, dodatne algoritme pretraživanja [3].

Dio s AI chatbotom pokazuje da umjetna inteligencija može pomoći u objašnjavanju pojmova, ali je nužno kritički provjeravati sadržaj i izvore.

Popis literature

- [1] jgsogo, »Sokoban: Prolog solver of Sokoban puzzle,« [Mrežno]. Available: <https://github.com/jgsogo/sokoban>. [Pokušaj pristupa 15 siječanj 2026].
- [2] N. Zhou i A. Dovier, »A tabled Prolog program for solving Sokoban,« *Science of Computer Programming*, svez. 75, 2010.
- [3] morenod, »Sokoban,« [Mrežno]. Available: <https://github.com/morenod/sokoban>. [Pokušaj pristupa 15 siječanj 2026].
- [4] A. Del Arte, »The basics of Sokoban,« 25 srpanj 2020. [Mrežno]. Available: <https://alonso-delarte.medium.com/the-basics-of-sokoban-f503a21e38ba>. [Pokušaj pristupa 15 siječanj 2026].
- [5] A. Del arte, »The basics of Sokoban level formats for designing your own Sokoban levels,« 2020. [Mrežno]. Available: <https://alonso-delarte.medium.com/the-basics-of-sokoban-level-formats-for-designing-your-own-sokoban-levels-51882a7a36f0>. [Pokušaj pristupa 15 siječanj 2026].
- [6] Reeborg's World, »Sokoban documentation,« [Mrežno]. Available: <https://reeborg.ca/docs/en/reference/sokoban.html>. [Pokušaj pristupa 15 siječanj 2026].
- [7] Rosetta Code, »Sokoban,« [Mrežno]. Available: <https://rosettacode.org/wiki/Sokoban>. [Pokušaj pristupa 15 siječanj 2026].
- [8] Google, »Gemini AI,« [Mrežno]. Available: <https://gemini.google.com/share/8620d0898462>. [Pokušaj pristupa 15 siječanj 2026].

Prilog A – Izvorni kod Prolog rješenja

U ovom prilogu prikazan je kod Prolog rješenja preuzet iz javno dostupnog GitHub repozitorija [1].

A.1. : sokoban.pl

```
/* Available directions */
direction(up).
direction(down).
direction(left).
direction(right).
direction(cenit).

/* Opposite directions */
opposite_dir(up, down).
opposite_dir(down, up).
opposite_dir(left, right).
opposite_dir(right, left).

/* Adjacent positions given a direction */
neib(P1, P2, up) :- top(P1, P2).
neib(P1, P2, down) :- top(P2, P1).
neib(P1, P2, right) :- right(P1, P2).
neib(P1, P2, left) :- right(P2, P1).

/* A corner is a square which has at most one neighbour square in vertical */
/* and one in horizontal. It is defined using the counter-rule noncorner. */
corner(X) :- \+ noncorner(X).
```

```
noncorner(X) :- top(_,X),top(X,_).
```

```
noncorner(X) :- right(_,X),right(X,_).
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
% Unit tests
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
:- begin_tests(board).
```

```
    %test(bottom) :- top(x1y3, x1y2), bottom(x1y2, x1y3).
```

```
:- end_tests(board).
```

A.2. : game.pl

```
final_state(sokoban, state(_Sokoban, Boxes)) :-
```

```
    all_boxes_in_solution(Boxes), !.
```

```
/* All boxes are in a the solution position when each one of them is at a */
```

```
/* solution position. */
```

```
all_boxes_in_solution([]).
```

```
all_boxes_in_solution([Box|Boxes]) :-
```

```
    solution(Box),
```

```
    all_boxes_in_solution(Boxes).
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
/* Game rules: positions to avoid */
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```

/* Some position for boxes must be avoided (unless they are solutions) */
/* because the Sokoban won't be able to move them further: */
/* - corners: the Sokoban can't move a box which is placed at a corner. */

stuck(X) :-
    \+ solution(X),
    corner(X).

/* - horizontally adjacent boxes can only be moved up or down... if there */
/* are empty squares for the Sokoban to push and for the boxes to move. */

stuck(X, Y) :-
    (right(X,Y); right(Y,X)),
    (\+ solution(X); \+ solution(Y)),
    (\+ top(X,_); \+ top(_,X)),
    (\+ top(Y,_); \+ top(_,Y)).

/* - vertically adjacent boxes can only be moved right or left... if there*/
/* are empty squares for the Sokoban to push and for the boxes to move. */

stuck(X, Y) :-
    (top(X,Y); top(Y,X)),
    (\+ solution(X); \+ solution(Y)),
    (\+ right(X,_); \+ right(_,X)),
    (\+ right(Y,_); \+ right(_,Y)).

/*****/

/* Game rules: move selection */

```

```

/*****/

/* The Sokoban can move to any empty position in the board, but cannot go */
/* through boxes. */
can_reach(P1, P1, _Boxes, _Visited).
can_reach(P1, P2, Boxes, _Visited) :-
    neib(P1, P2, _),
    \+ member(P2, Boxes).
can_reach(P1, P2, Boxes, Visited) :-
    neib(P1, P3, _),
    P3 \== P2,
    \+ member(P3, Visited),
    \+ member(P3, Boxes),
    can_reach(P3, P2, Boxes, [P3|Visited]).

/* A good place to move a box is one that: */
/* - is not already occupied by a box. */
/* - is not one of the positions to avoid regarding the board and boxes. */
good_move(X, Boxes) :-
    \+ member(X, Boxes),
    \+ stuck(X),
    foreach(member(Box, Boxes), \+ stuck(X, Box)).

/* Selection of a good movement given a state: */
/* - any valid movement for every box */
/* - the Sokoban must be able to access the push position */

```



```

movement(state(Sokoban, Boxes), move(Box, Dir)) :-
    select(Box, Boxes, BoxesRemain),
    neib(Box, NextLoc, Dir),
    good_move(NextLoc, BoxesRemain),
    neib(PushPosition, Box, Dir),

    /*can_reach(Sokoban, PushPosition, Boxes, []),*/ /* Enable this rule to consider Sokoban
movement constraint */

    \+ member(PushPosition, Boxes).

```

```

/*****

/* Implementation of the state update functionality.          */

*****/

```

```

update(state(_Sokoban, Boxes), move(Box, Dir), state(NewSokoban, NewBoxes)) :-
    NewSokoban = Box,
    subtract(Boxes, [Box], TempList),
    neib(Box, NewPosition, Dir),
    append(TempList, [NewPosition], NewBoxes).

```

```

%%%%%%%%%%%%%%

```

```

% Unit tests

```

```

%%%%%%%%%%%%%%

```

```

:- begin_tests(game).

```

```

:- end_tests(game).

```

A.3. : board.pl

```

/* Available directions                                     */

```

direction(up).

direction(down).

direction(left).

direction(right).

direction(cenit).

/* Opposite directions */

opposite_dir(up, down).

opposite_dir(down, up).

opposite_dir(left, right).

opposite_dir(right, left).

/* Adjacent positions given a direction */

neib(P1, P2, up) :- top(P1, P2).

neib(P1, P2, down) :- top(P2, P1).

neib(P1, P2, right) :- right(P1, P2).

neib(P1, P2, left) :- right(P2, P1).

/* A corner is a square which has at most one neighbour square in vertical */

/* and one in horizontal. It is defined using the counter-rule noncorner. */

corner(X) :- \+ noncorner(X).

noncorner(X) :- top(_,X),top(X,_).

noncorner(X) :- right(_,X),right(X,_).

%%

% Unit tests

%%

:- begin_tests(board).

 %test(bottom) :- top(x1y3, x1y2), bottom(x1y2, x1y3).

:- end_tests(board).

A.4. : level1.pl

/* Board definition: adjacent positions */

top(x1y1,x1y2).

top(x1y2,x1y3).

top(x2y1,x2y2).

top(x2y2,x2y3).

top(x3y1,x3y2).

top(x3y2,x3y3).

right(x1y3, x2y3).

right(x2y3, x3y3).

right(x1y2, x2y2).

right(x2y2, x3y2).

right(x1y1, x2y1).

right(x2y1, x3y1).

/* Boxes initial positions */

box(x2y2).

box(x3y2).

/* Storage positions */

```
solution(x2y1).
```

```
solution(x3y1).
```

```
/* Sokoban initial position */
```

```
sokoban(x1y1).
```

```
/* Initial state assertion */
```

```
initial_state(sokoban, state(x1y1, [x2y2, x3y2])).
```

Prilog B – Izvorni kod Python rješenja

Ovaj prilog sadrži Python rješenje preuzet iz javno dostupnog GitHub repozitorija [3].

```
import sys
import pygame
import string
import queue

class game:

    def is_valid_value(self,char):
        if ( char == ' ' or #floor
            char == '#' or #wall
            char == '@' or #worker on floor
            char == '.' or #dock
            char == '*' or #box on dock
            char == '$' or #box
            char == '+' ): #worker on dock
            return True
        else:
            return False

    def __init__(self,filename,level):
        self.queue = queue.LifoQueue()
        self.matrix = []
#         if level < 1 or level > 50:
            if level < 1:
                print ("ERROR: Level "+str(level)+" is out of range")
                sys.exit(1)
            else:
                file = open(filename,'r')
                level_found = False
                for line in file:
                    row = []
                    if not level_found:
```

```

        if "Level "+str(level) == line.strip():
            level_found = True
        else:
            if line.strip() != "":
                row = []
                for c in line:
                    if c != '\n' and self.is_valid_value(c):
                        row.append(c)
                    elif c == '\n': #jump to next row when newline
                        continue
                    else:
                        print ("ERROR: Level "+str(level)+" has invalid value "+c)
                        sys.exit(1)
                self.matrix.append(row)
            else:
                break

def load_size(self):
    x = 0
    y = len(self.matrix)
    for row in self.matrix:
        if len(row) > x:
            x = len(row)
    return (x * 32, y * 32)

def get_matrix(self):
    return self.matrix

def print_matrix(self):
    for row in self.matrix:
        for char in row:
            sys.stdout.write(char)
            sys.stdout.flush()
        sys.stdout.write('\n')

def get_content(self,x,y):
    return self.matrix[y][x]

```

```

def set_content(self,x,y,content):
    if self.is_valid_value(content):
        self.matrix[y][x] = content
    else:
        print ("ERROR: Value '"+content+"' to be added is not valid")

def worker(self):
    x = 0
    y = 0
    for row in self.matrix:
        for pos in row:
            if pos == '@' or pos == '+':
                return (x, y, pos)
            else:
                x = x + 1
        y = y + 1
    x = 0

def can_move(self,x,y):
    return self.get_content(self.worker()[0]+x,self.worker()[1]+y) not in ['#','*','$']

def next(self,x,y):
    return self.get_content(self.worker()[0]+x,self.worker()[1]+y)

def can_push(self,x,y):
    return (self.next(x,y) in ['*','$'] and self.next(x+x,y+y) in [' ','.'])

def is_completed(self):
    for row in self.matrix:
        for cell in row:
            if cell == '$':
                return False
    return True

def move_box(self,x,y,a,b):
#    (x,y) -> move to do

```

```

# (a,b) -> box to move
current_box = self.get_content(x,y)
future_box = self.get_content(x+a,y+b)
if current_box == '$' and future_box == ' ':
    self.set_content(x+a,y+b,'$')
    self.set_content(x,y,' ')
elif current_box == '$' and future_box == ' ':
    self.set_content(x+a,y+b,'*')
    self.set_content(x,y,' ')
elif current_box == '*' and future_box == ' ':
    self.set_content(x+a,y+b,'$')
    self.set_content(x,y,'.')
elif current_box == '*' and future_box == ' ':
    self.set_content(x+a,y+b,'*')
    self.set_content(x,y,'.')

def unmove(self):
    if not self.queue.empty():
        movement = self.queue.get()
        if movement[2]:
            current = self.worker()
            self.move(movement[0] * -1,movement[1] * -1, False)
            self.move_box(current[0]+movement[0],current[1]+movement[1],movement[0] * -
1,movement[1] * -1)
        else:
            self.move(movement[0] * -1,movement[1] * -1, False)

def move(self,x,y,save):
    if self.can_move(x,y):
        current = self.worker()
        future = self.next(x,y)
        if current[2] == '@' and future == ' ':
            self.set_content(current[0]+x,current[1]+y,'@')
            self.set_content(current[0],current[1], ' ')
            if save: self.queue.put((x,y,False))
        elif current[2] == '@' and future == ' ':
            self.set_content(current[0]+x,current[1]+y,'+')

```



```

        self.set_content(current[0],current[1],' ')
        if save: self.queue.put((x,y,False))
    elif current[2] == '+' and future == ' ':
        self.set_content(current[0]+x,current[1]+y,'@')
        self.set_content(current[0],current[1],'.')
        if save: self.queue.put((x,y,False))
    elif current[2] == '+' and future == '.':
        self.set_content(current[0]+x,current[1]+y, '+')
        self.set_content(current[0],current[1],'.')
        if save: self.queue.put((x,y,False))
elif self.can_push(x,y):
    current = self.worker()
    future = self.next(x,y)
    future_box = self.next(x+x,y+y)
    if current[2] == '@' and future == '$' and future_box == ' ':
        self.move_box(current[0]+x,current[1]+y,x,y)
        self.set_content(current[0],current[1],' ')
        self.set_content(current[0]+x,current[1]+y,'@')
        if save: self.queue.put((x,y,True))
    elif current[2] == '@' and future == '$' and future_box == '.':
        self.move_box(current[0]+x,current[1]+y,x,y)
        self.set_content(current[0],current[1],' ')
        self.set_content(current[0]+x,current[1]+y,'@')
        if save: self.queue.put((x,y,True))
    elif current[2] == '@' and future == '*' and future_box == ' ':
        self.move_box(current[0]+x,current[1]+y,x,y)
        self.set_content(current[0],current[1],' ')
        self.set_content(current[0]+x,current[1]+y, '+')
        if save: self.queue.put((x,y,True))
    elif current[2] == '@' and future == '*' and future_box == '.':
        self.move_box(current[0]+x,current[1]+y,x,y)
        self.set_content(current[0],current[1],' ')
        self.set_content(current[0]+x,current[1]+y, '+')
        if save: self.queue.put((x,y,True))
    if current[2] == '+' and future == '$' and future_box == ' ':
        self.move_box(current[0]+x,current[1]+y,x,y)
        self.set_content(current[0],current[1],'.')

```

```

        self.set_content(current[0]+x,current[1]+y,'@')
        if save: self.queue.put((x,y,True))
    elif current[2] == '+' and future == '$' and future_box == '.':
        self.move_box(current[0]+x,current[1]+y,x,y)
        self.set_content(current[0],current[1],'.')
        self.set_content(current[0]+x,current[1]+y, '+')
        if save: self.queue.put((x,y,True))
    elif current[2] == '+' and future == '*' and future_box == '.':
        self.move_box(current[0]+x,current[1]+y,x,y)
        self.set_content(current[0],current[1],'.')
        self.set_content(current[0]+x,current[1]+y, '+')
        if save: self.queue.put((x,y,True))
    elif current[2] == '+' and future == '*' and future_box == '.':
        self.move_box(current[0]+x,current[1]+y,x,y)
        self.set_content(current[0],current[1],'.')
        self.set_content(current[0]+x,current[1]+y, '+')
        if save: self.queue.put((x,y,True))

```

```

def print_game(matrix,screen):
    screen.fill(background)
    x = 0
    y = 0
    for row in matrix:
        for char in row:
            if char == '.': #floor
                screen.blit(floor,(x,y))
            elif char == '#': #wall
                screen.blit(wall,(x,y))
            elif char == '@': #worker on floor
                screen.blit(worker,(x,y))
            elif char == '.': #dock
                screen.blit(docker,(x,y))
            elif char == '*': #box on dock
                screen.blit(box_docked,(x,y))
            elif char == '$': #box
                screen.blit(box,(x,y))
            elif char == '+': #worker on dock

```

```

        screen.blit(worker_docked,(x,y))
    x = x + 32
    x = 0
    y = y + 32

def get_key():
    while 1:
        event = pygame.event.poll()
        if event.type == pygame.KEYDOWN:
            return event.key
        else:
            pass

def display_box(screen, message):
    "Print a message in a box in the middle of the screen"
    fontobject = pygame.font.Font(None,18)
    pygame.draw.rect(screen, (0,0,0),
        ((screen.get_width() / 2) - 100,
         (screen.get_height() / 2) - 10,
         200,20), 0)
    pygame.draw.rect(screen, (255,255,255),
        ((screen.get_width() / 2) - 102,
         (screen.get_height() / 2) - 12,
         204,24), 1)
    if len(message) != 0:
        screen.blit(fontobject.render(message, 1, (255,255,255)),
            ((screen.get_width() / 2) - 100, (screen.get_height() / 2) - 10))
    pygame.display.flip()

def display_end(screen):
    message = "Level Completed"
    fontobject = pygame.font.Font(None,18)
    pygame.draw.rect(screen, (0,0,0),
        ((screen.get_width() / 2) - 100,
         (screen.get_height() / 2) - 10,
         200,20), 0)
    pygame.draw.rect(screen, (255,255,255),

```

```

        ((screen.get_width() / 2) - 102,
         (screen.get_height() / 2) - 12,
         204,24), 1)
    screen.blit(fontobject.render(message, 1, (255,255,255)),
                ((screen.get_width() / 2) - 100, (screen.get_height() / 2) - 10))
    pygame.display.flip()

```

```

def ask(screen, question):
    "ask(screen, question) -> answer"
    pygame.font.init()
    current_string = []
    display_box(screen, question + ": " + "".join(current_string))
    while 1:
        inkey = get_key()
        if inkey == pygame.K_BACKSPACE:
            current_string = current_string[0:-1]
        elif inkey == pygame.K_RETURN:
            break
        elif inkey == pygame.K_MINUS:
            current_string.append("_")
        elif inkey <= 127:
            current_string.append(chr(inkey))
        display_box(screen, question + ": " + "".join(current_string))
    return "".join(current_string)

```

```

def start_game():
    start = pygame.display.set_mode((320,240))
    level = ask(start,"Select Level")
    level = int(level)
    if level > 0:
        return level
    else:
        print ("ERROR: Invalid Level: ")+str(level)
        sys.exit(2)

```

```

wall = pygame.image.load('images/wall.png')
floor = pygame.image.load('images/floor.png')

```

```

box = pygame.image.load('images/box.png')
box_docked = pygame.image.load('images/box_docked.png')
worker = pygame.image.load('images/worker.png')
worker_docked = pygame.image.load('images/worker_dock.png')
docker = pygame.image.load('images/dock.png')
background = 255, 226, 191
pygame.init()

level = start_game()
game = game('levels',level)
size = game.load_size()
screen = pygame.display.set_mode(size)
while 1:
    if game.is_completed(): display_end(screen)
    print_game(game.get_matrix(),screen)
    for event in pygame.event.get():
        if event.type == pygame.QUIT: sys.exit(0)
        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_UP: game.move(0,-1, True)
            elif event.key == pygame.K_DOWN: game.move(0,1, True)
            elif event.key == pygame.K_LEFT: game.move(-1,0, True)
            elif event.key == pygame.K_RIGHT: game.move(1,0, True)
            elif event.key == pygame.K_q: sys.exit(0)
            elif event.key == pygame.K_d: game.unmove()
    pygame.display.update()

```